form to upload file data is to create this type of request, so this is what we'll look at here. (We'll be examining the other side of this equation when we take a look at HTML in the next section.)

A client that is capable of multipart requests (any web browser, or even the curl binary) will split the request into multiple parts separated by a *boundary string*; each part has its own headers and body data. This will make more sense if we look at some practical examples, so let's upload a file first. The behavior around POST uploaded files is controlled by a number of `.ini` settings, detailed in the following table.

Table 11-2. *PHP `ini` directives for uploads*

Directive	Default	
`file_uploads`	`"1"`	Allow (or not) file uploads via POST and PUT
`upload_max_filesize`	`"2M"`	Maximum size of an uploaded file must be larger than `post_max_size`
`upload_temp_dir`	`null`	Location for temporary uploads (PHP will use system default if null)
`post_max_size`		Maximum size (Content-Length) of POST data
`max_input_time`	`"-1"`	Maximum time allowed to parse request data

To get started, let's just have a very simple look at the `$_FILES` superglobal that PHP creates when handling file uploads. Create yet another `index.php` file like this:

```
// index.php

var_dump($_FILES);
```

Then use the following curl command, which assumes that a `./random.jpg` file is available in the current working directory (otherwise curl will exit with an error message about failing to "open/read local data"). Any old data will do; it just needs to exist in the correct place on your local filesystem. All being well, you'll see the following output from the PHP handler:

```
$ curl -v -F 'file=@./random.jpg' http://localhost:8080/index.php
* Host localhost:8080 was resolved.
...
> Content-Length: 612379
```